

JAVA PROGRAMMING · DAY 5

Nested Patterns & Arrays

Number/Star Patterns, Array Basics, Float & String Arrays, Odd-Even Sum, Product of Array

7 Programs · Iteration-by-Iteration Explanations

1 Pattern Printing — Descending Number Triangle

Just like the star patterns from Day 4, number patterns use the same nested-loop skeleton — only now the inner loop prints a **counting value** instead of a fixed symbol.

Program 1 — Countdown Triangle (10 down to row number)

```
public class Main {
    public static void main(String[] args) {
        for (int i = 1; i <= 10; i++) {
            for (int j = 10; j >= i; j--) {
                System.out.print(j + " ");
            }
            System.out.println();
        }
    }
}
```

CONSOLE OUTPUT

```
10 9 8 7 6 5 4 3 2 1
10 9 8 7 6 5 4 3 2
10 9 8 7 6 5 4 3
10 9 8 7 6 5 4
10 9 8 7 6 5
10 9 8 7 6
10 9 8 7
10 9 8
10 9
10
```

Iteration Trace — Outer i picks the row, Inner j counts down from 10 to i

Outer i	Inner j runs from 10 down to...	Numbers Printed
1	10 → 1	10 9 8 7 6 5 4 3 2 1
2	10 → 2	10 9 8 7 6 5 4 3 2
3	10 → 3	10 9 8 7 6 5 4 3
...	...	...
10	10 → 10	10

What's happening here?

- The inner loop always **starts** at 10, but its **stopping point** (`j >= i`) changes with every row.
- As `i` grows from 1 to 10 (row 1, row 2...), the inner loop has less distance to cover, so each row prints **one fewer number** than the row before it.
- The inner loop counts **downwards** this time (`j--`), which is why the numbers appear largest-to-smallest on each row.

2 Pattern Printing — Right-Aligned Star Pyramid

To make a shape lean to one side, we print **spaces first**, then the symbols. This needs **two inner loops** inside one outer loop — one for the spaces, one for the stars.

Program 1 — Right-Aligned Star Pyramid

Expected Output (spaces align the stars to the right):

```
public class Main {
    public static void main(String[] args) {
        int n = 5;
        for (int i = 1; i <= n; i++) {
            for (int space = 1; space <= n - i; space++) {
                System.out.print(" ");
            }
            for (int star = 1; star <= i; star++) {
                System.out.print("*");
            }
            System.out.println();
        }
    }
}
```

CONSOLE OUTPUT

```

  *
 **
***
****
*****
```

Iteration Trace

Outer i (row)	Spaces (n-i)	Stars (i)	Row Output
1	4	1	*
2	3	2	**
3	2	3	***
4	1	4	****
5	0	5	*****

What's happening here?

- Each row prints **spaces first** (using the first inner loop), then **stars** (using the second inner loop) — both loops run one after another, on the same row, before `println()` moves to the next line.
- The number of spaces ($n - i$) **shrinks** as i grows, while the number of stars (i) **grows** — together they always add up to 5, keeping the right edge aligned.

3 Arrays — Storing Multiple Values

So far, each variable could hold only **one** value. An **array** is a single variable that can hold **many values of the same type**, stored in order, and accessed using a position number called an **index**.

Program 1 — Declare, Store and Print an int Array

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        int arr[] = {1, 2, 3, 4, 5};
        for (int i = 0; i < arr.length; i++) {
            System.out.print(arr[i] + " ");
        }
    }
}
```

CONSOLE OUTPUT

```
1 2 3 4 5
```

How the array is stored (index starts at 0!)

1	2	3	4	5
[0]	[1]	[2]	[3]	[4]

What's happening here?

- **int arr[] = {1,2,3,4,5};** creates an array holding 5 numbers. (You'll also commonly see this written as **int[] arr = {...}** — both mean the same thing.)
- **Indexing starts at 0**, not 1. So **arr[0]** is the first value (1), and **arr[4]** is the last value (5) — not arr[5]!
- **arr.length** gives the total count of items (5 here). Using it in the loop condition means the loop always works correctly, even if you resize the array later.

4 Marks Array — float Data Type

Arrays aren't limited to whole numbers. A **float** array is perfect for storing marks or scores that may include decimals.

Program 1 — Store and Print Marks (float array)

Values: 99.00, 77.00, 88.00, 66.00, 77.00, 88.00, 55.00

```
public class Main {
    public static void main(String[] args) {
        float marks[] = {99.00f, 77.00f, 88.00f, 66.00f, 77.00f, 88.00f, 55.00f};
        for (int i = 0; i < marks.length; i++) {
            System.out.print(marks[i] + " ");
        }
    }
}
```

CONSOLE OUTPUT

```
99.0 77.0 88.0 66.0 77.0 88.0 55.0
```

What's happening here?

- Notice the **f** at the end of every number (99.00f). In Java, a plain decimal like 99.00 is automatically treated as a **double**, not a **float** — the **f** is required to tell Java "store this as a float".
- Same loop pattern as the int array — only the data type changed from **int** to **float**.

5 Subject Array — String Data Type

Arrays can also hold **text**. A String array stores a list of words, each accessible by its index — just like the number arrays.

Program 1 — Store and Print Subjects (String array)

Values: "Maths", "English", "Science", "Social", "Tamil"

```
public class Main {  
    public static void main(String[] args) {  
        String subjects[] = {"Maths", "English", "Science", "Social", "Tamil"};  
        for (int i = 0; i < subjects.length; i++) {  
            System.out.print(subjects[i] + " ");  
        }  
    }  
}
```

CONSOLE OUTPUT

```
Maths English Science Social Tamil
```

What's happening here?

- Each element is wrapped in **double quotes** because they're text (String), not numbers.
- **subjects[0]** is "Maths", **subjects[4]** (the last one, since length is 5) is "Tamil".

6 Sum of Odd and Even Numbers in an Array

Combining an array loop with an **if** condition (just like Day 4) lets us split and total values based on a rule.

Program 1 — Separate Sum of Even and Odd Numbers

Array: {1,2,3,4,5,6,7,8,9,10}

```
public class Main {
    public static void main(String[] args) {
        int numbers[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
        int evenSum = 0;
        int oddSum = 0;

        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] % 2 == 0) {
                evenSum += numbers[i];
            } else {
                oddSum += numbers[i];
            }
        }

        System.out.println("Sum of even numbers = " + evenSum);
        System.out.println("Sum of odd numbers = " + oddSum);
    }
}
```

CONSOLE OUTPUT

```
Sum of even numbers = 30
Sum of odd numbers = 25
```

Iteration Trace

Index i	numbers[i]	Even or Odd?	evenSum	oddSum
0	1	Odd	0	1
1	2	Even	2	1
2	3	Odd	2	4
3	4	Even	6	4
4	5	Odd	6	9
5	6	Even	12	9
6	7	Odd	12	16
7	8	Even	20	16
8	9	Odd	20	25
9	10	Even	30	25

What's happening here?

- We use **two separate accumulator variables**, evenSum and oddSum, both starting at 0.
- The loop visits every index from 0 to numbers.length - 1, checks each value with % 2, and adds it to the matching total.
- By the last row of the trace table, every number has been sorted into one of the two totals — that's the final answer.

7 Product ("Multiple") of All Elements in an Array

Just like we used **sum += ...** to add values, we can use **product *= ...** to multiply them all together.

Program 1 — Multiply All Array Elements

Array: {1,2,3,4,5,6,7,8,9,10} | **Expected:** 1×2×3×...×10

```
public class Main {
    public static void main(String[] args) {
        int numbers[] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
        long product = 1;

        for (int i = 0; i < numbers.length; i++) {
            product *= numbers[i];
        }

        System.out.println("Product of all elements = " + product);
    }
}
```

CONSOLE OUTPUT

```
Product of all elements = 3628800
```

Iteration Trace

Index i	numbers[i]	product after (product *= numbers[i])
0	1	1
1	2	2
2	3	6
3	4	24
4	5	120
5	6	720
6	7	5040
7	8	40320
8	9	362880
9	10	3628800

What's happening here?

- **product** starts at **1**, not 0 — starting a product accumulator at 0 would make everything multiply to zero!
- **product *= numbers[i];** is shorthand for **product = product * numbers[i];**
- We used **long** instead of **int** for product as good practice — products grow very fast, and **long** can safely hold much bigger results than **int** before overflowing.



Practice Questions — Master Arrays

Try these on your own using the same loop patterns from this PDF. No solutions given — that's the point!

1 Find the largest and smallest number in an array.

Hint: start with two variables set to `arr[0]`, then compare every element against them inside the loop.

2 Reverse the elements of an array and print them.

Hint: loop the index backwards, from `arr.length - 1` down to `0`.

3 Count how many marks in the float marks array (Topic 4) are ≥ 80 .

Hint: combine the array loop with an `if`, like you did for odd/even sum.

4 Find the average (mean) of all elements in the numbers array (Topic 6).

Hint: reuse your sum logic, then divide by `numbers.length`. Watch out for integer division!

5 Search for a specific subject (e.g. "Science") in the subjects array and print its index.

Hint: loop through the array and compare each element using `.equals()`, not `==`, for Strings.

6 Count how many vowels appear across all subject names combined.

Hint: this combines Day 4's vowel-checking idea with looping through each String's characters — a nested loop!

7 Print the numbers array (Topic 6) in reverse using a right-aligned pattern style, one number fewer per row.

Hint: combine Topic 1's pattern loop with array indexing instead of a plain counter.

→ Coming Up Next

A quick look at what's ahead in the course, building directly on today's array foundations.

DAY 6 Array-Based Programs

Searching (linear search), sorting basics (bubble sort), finding duplicates, and merging two arrays.

DAY 7 2D Arrays (Matrices)

Rows and columns, nested-loop traversal, matrix addition, and printing grid patterns.

DAY 8 Strings & String Methods

length(), charAt(), substring(), toUpperCase()/toLowerCase(), and building your own String-processing programs.

DAY 9 Methods / Functions

Writing reusable blocks of code with parameters and return values, replacing repeated logic with a single method call.

DAY 10 Introduction to OOP — Classes & Objects

Moving beyond a single main class: defining your own classes, creating objects, and understanding constructors.

End of Day 5 — Nested Number/Star Patterns & Java Arrays